

[Open in app](#)Access to everything. Now 30% off. [Upgrade now](#)

Homography



Computer Vision



Robotics



Image Processing



Image Transformation



Homography

4 min read · Mar 22, 2023



Shantanuparab

[Follow](#)

Listen



Share



More

Homography is an essential concept in image processing and finds its use in many other fields.

Homography is a transformation matrix that takes you from one plane to another plane. In this project, we will learn about homography and explore one of its applications. We will place a logo of our choice on a field goal post using Homography.



Video nicht verfügbar

Dieses Video ist nicht verfügbar

Video Demonstration of Project

Consider the below image of a square that is transformed into another orientation. Such orientation can be defined as a combination of mathematical operations such as rotation, translation, skewing, scaling, etc. These operations can be described as matrices as follows.

Translation: This is an operation that shifts all the points as defined in X or Y direction. We use the following 3×3 matrix to transform this to this.

$$T = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotation: In the following example we rotate the square by 45. We use a 2×2 matrix as follows

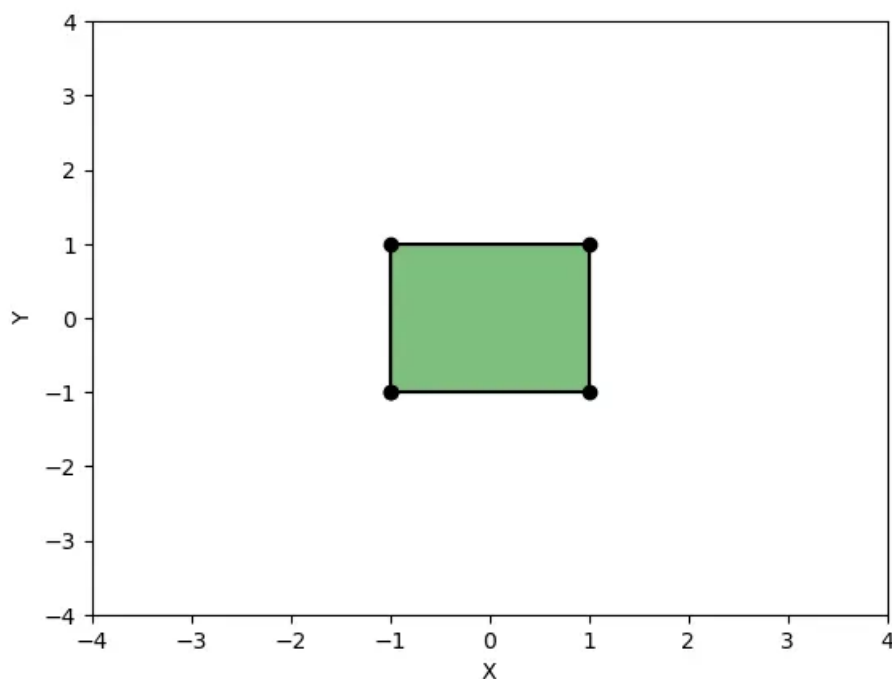
$$R = \begin{bmatrix} \cos 45 & -\sin 45 \\ \sin 45 & \cos 45 \end{bmatrix}$$

Skew: Here we perform a uniform skew operation and use a 2×2 matrix as follows

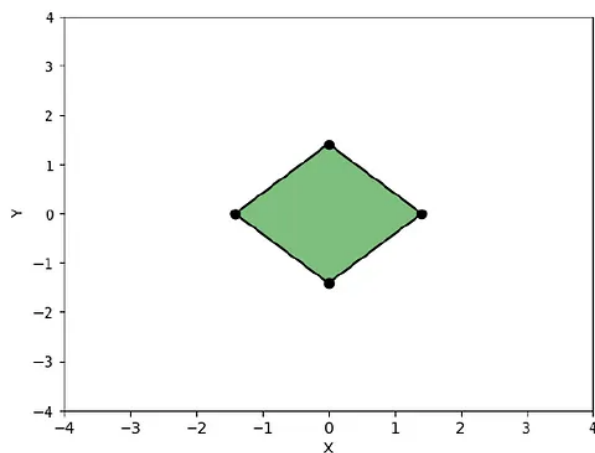
$$Sk = \begin{bmatrix} 1 & 0.25 \\ 0.25 & 1 \end{bmatrix}$$

Scaling: In this example, we scale the image 2 times its original size. We use a 2×2 matrix as follows

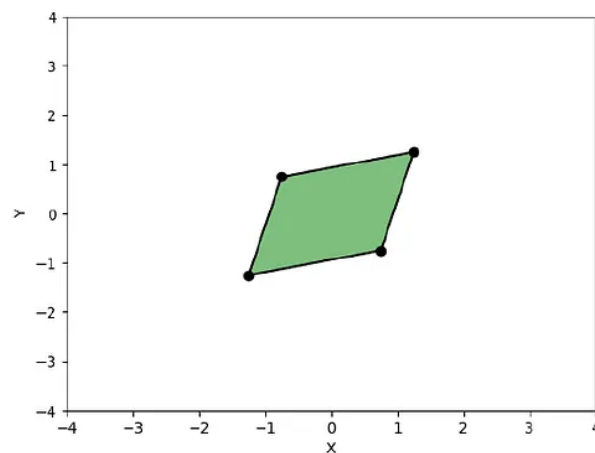
$$Sc = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$



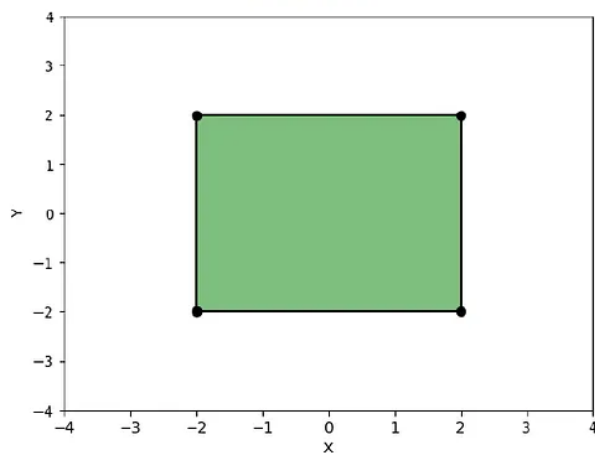
Original Image of Square



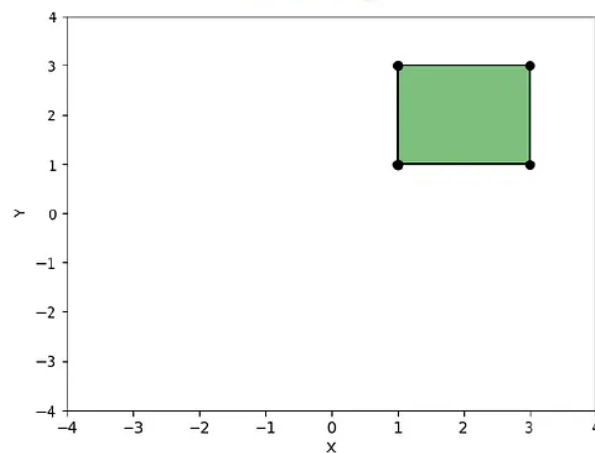
Rotation



Skewing



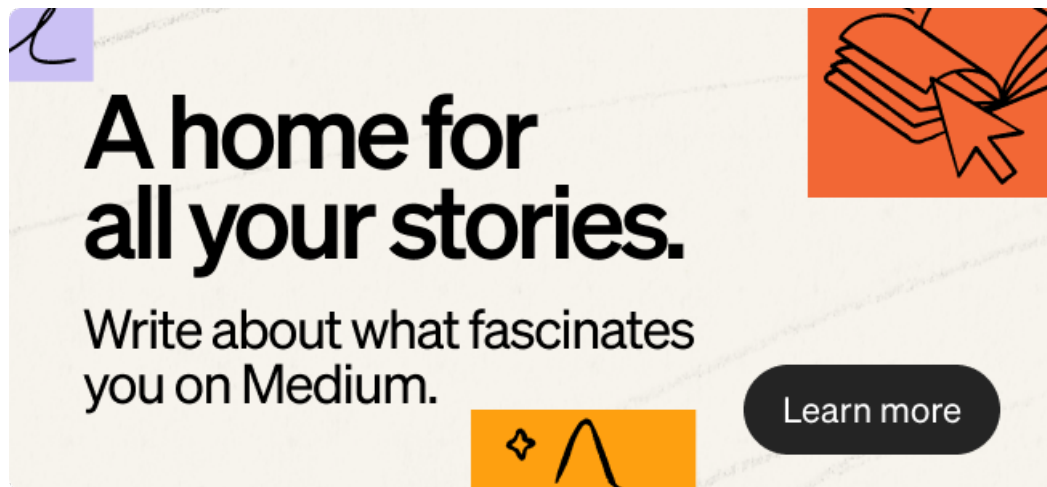
Scaling



Translation

Matrix Transformations

We see that rotation, scaling, and skewing in 2D space can be described as a 2×2 matrix. These are known as affine transformations. However to depict translation we need another row as shown above. When combined together these 3×3 matrices can transform an image from one perspective into another. Such matrices are very useful in our further demonstration. When we have to find out how a particular feature of the source image transforms to go to the destination image we try to find out this transformation matrix known as Homography.



The homography matrix can be calculated as follows. Let us consider points x_s and y_s as points in the source images and x_d and y_d as points in the destination image. Now as discussed above the relation between these points is given as a matrix multiplication.

$$\begin{bmatrix} x_d \\ y_d \\ 1 \end{bmatrix} = \begin{bmatrix} \bar{x}_d \\ \bar{y}_d \\ \bar{z}_d \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ 1 \end{bmatrix}$$

Now that we have established a relation between the source and destination points we need a way to find out the homography matrix so that we can use it to perform various tasks such as image stitching, image projection, image stabilization, etc. We have 9 unknowns but we have 8 degrees of freedom as the homography matrix is estimated up to a scale. We need a minimum of 4 points to compute the homography however we always try to get more points to get a better estimate. The above equation can be re-written as follows:

$$x_d^n = \frac{\bar{x}_d^n}{\bar{z}_d^n} = \frac{h_{11}x_s^n + h_{12}y_s^n + h_{13}}{h_{31}x_s^n + h_{32}y_s^n + h_{33}}$$

$$y_d^n = \frac{\bar{y}_d^n}{\bar{z}_d^n} = \frac{h_{21}x_s^n + h_{22}y_s^n + h_{23}}{h_{31}x_s^n + h_{32}y_s^n + h_{33}}$$

Here n is the number of pairs in the images that is x_s, y_s and x_d, y_d . We can rearrange the equations and

write them as linear equations as follows

$$x_d^n(h_{31}x_s^n + h_{32}y_s^n + h_{33}) = h_{11}x_s^n + h_{12}y_s^n + h_{13} \quad (1)$$

$$y_d^n(h_{31}x_s^n + h_{32}y_s^n + h_{33}) = h_{21}x_s^n + h_{22}y_s^n + h_{23} \quad (2)$$

$$\begin{bmatrix} x_s^n & y_s^n & 1 & 0 & 0 & 0 & -x_d^n x_s^n & -x_d^n y_s^n & -x_d^n \\ 0 & 0 & 0 & x_s^n & y_s^n & 1 & -y_d^n x_s^n & -y_d^n y_s^n & -y_d^n \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (3)$$

We cannot solve the above equation with only two pairs of points so we stack up matching pairs of points to create an over-deterministic system as follows.

$$\begin{bmatrix} x_s^1 & y_s^1 & 1 & 0 & 0 & 0 & -x_d^1 x_s^1 & -x_d^1 y_s^1 & -x_d^1 \\ 0 & 0 & 0 & x_s^1 & y_s^1 & 1 & -y_d^1 x_s^1 & -y_d^1 y_s^1 & -y_d^1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_s^n & y_s^n & 1 & 0 & 0 & 0 & -x_d^n x_s^n & -x_d^n y_s^n & -x_d^n \\ 0 & 0 & 0 & x_s^n & y_s^n & 1 & -y_d^n x_s^n & -y_d^n y_s^n & -y_d^n \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ \vdots \\ \vdots \\ \vdots \\ 0 \\ 0 \end{bmatrix} \quad (4)$$

Let the matrix of the known variables be A and the matrix of the unknowns be h. So we can write

$$Ah = 0 \text{ such that } ||h||^2 = 1$$

We solve this using the constrained least square method which is defined as

$$\min_h ||Ah||^2 \text{ such that } ||h||^2 = 1$$

We know that

$$\|Ah\|^2 = (Ah)^T(Ah) = h^T A^T Ah \text{ and } \|h\|^2 = h^T h = 1$$

We want to find h that minimizes $h^T A^T Ah$ such that $h^T h = 1$. In order to achieve this we define a loss function.

$$L(h, \lambda) = h^T A^T Ah - \lambda(h^T h - 1) \quad (5)$$

Taking derivatives of $L(h, \lambda)$ w.r.t h we get

$$2A^T Ah - 2\lambda h = 0 \quad (6)$$

$$A^T Ah = \lambda h \quad (7)$$

This is an eigenvalue problem. The solution to the problem is the eigenvector h with the smallest eigenvalue λ of matrix $A^T A$, which minimizes the loss function. We then reshape this vector into a 3×3 matrix to give us the homography matrix between the set of points.

```
def homography(source_points, destination_points):
    A=[]
    for i in range(source_points.shape[0]):
        x,y=source_points[i,0],source_points[i,1]
        xw,yw=destination_points[i,0],destination_points[i,1]
        A.append([x,y,1,0,0,0,-xw*x,-xw*y,-xw])
        A.append([0,0,0,x,y,1,-yw*x,-yw*y,-yw])
    A=np.array(A)
    eigenvalues, eigenvectors = np.linalg.eig(A.T@A)
    min_eig_idx=np.argmin(eigenvalues)
    smallest_eigen_vector=eigenvectors[:,min_eig_idx]
    H=np.reshape(smallest_eigen_vector,(3,3))
    H=H/H[2,2]
    return H
```

We first start by creating a matrix A of known variables as shown in Equation 4. We then find the eigenvector corresponding to the smallest eigenvalue and reshape it into a 3×3 matrix. We finally normalize the matrix to get the homography matrix. We will be using this function in our application of homography.

We will be demonstrating the use of Homography in our small project where we will place a logo on a football field post. The destination image and the logo are as given below.



(a) Goal Post



(b) Logo

Destination and Source Image

To accomplish our goal we consider the source points as the corners of the logo and the destination points will be the four corners of the field goalas shown below.



We will be computing Homography between the source and destination. This homography matrix will be used to transform the perspective of the logo to fit into the goal post. Then we will blend the two images using some image processing techniques. The final output is as shown below.



You can refer to the code for above project here

<https://github.com/shantanuparabumd/Homography.git>

Homography



Computer Vision



Robotics



Image Processing



Image Transformation



Follow

Written by Shantanuparab

28 followers · 1 following

Responses (1)



Felix

What are your thoughts?



Climb
Jul 29



$h_{31} * x_s + h_{32} + h_{33}$ maybe is $h_{31} * x_s + h_{32} * y_s + h_{33}$?



[Reply](#)

